How a Unified Development Toolchain Supports Automated Testing Consistency in a TypeScript Full-Stack Monorepo: A Case Study of Circles

Aldiyar Seylkhanov
Bachelor Degree Student, School of Software Engineering
Astana IT University (AITU)
Astana, Kazakhstan
seylkhanov.aldiyar@gmail.com

Alexandr Tyulkov
Master Degree Student, School of Software Engineering
Astana IT University (AITU)
Astana, Kazakhstan
widesehl@gmail.com
0009-0009-6422-0559

***Abstract*—TypeScript monorepos often combine frontend, backend, and shared packages under one repository, but test execution is still shaped by package-local configuration. This paper studies how a unified toolchain affects testing consistency in such a setting. The Circles repository is used as a case study because it runs backend, frontend, and shared-package tests through the Vite+ command interface while still keeping a small number of package-specific overrides. The evaluation uses repository inspection, CI workflow analysis, and direct execution of test and coverage commands. The observed configuration surface consists of nine testing and CI configuration files, seven automated test files, and two GitHub Actions workflows. Empirical results show stable execution in the backend and shared package, with 21 backend tests and one shared-package test passing, while the aggregated frontend run exposes a configuration-level failure that does not appear when the frontend unit project is executed in isolation. This result is important because it shows that a unified command surface improves consistency, but multi-project composition can still preserve hidden environment differences. The paper therefore argues that unified toolchains reduce configuration fragmentation and improve reproducibility, yet consistency must still be evaluated at the combined pipeline level rather than only at the per-project level.**

***Index Terms*—risk-based testing, test automation, TypeScript, monorepo, CI/CD, quality gates, edge-case testing**

I. Introduction

Risk-based testing (RBT) allocates effort proportionally to the probability and impact of failure for each system component. Applied to a production TypeScript monorepo, this strategy requires two inputs that are unavailable at planning time: observed defect data and real coverage measurements. The initial planning phase produces a risk model built on assumptions. The subsequent baseline automation study delivers an initial test suite and pipeline. This paper reports a third iteration: it closes the feedback loop by re-evaluating the model using empirical evidence, extending the suite to probe the original assumptions, and documenting what the plan missed.

This study covers the Circles project, a full-stack TypeScript monorepo consisting of a React frontend, a Hono REST API backend, and shared utility packages. The system is built and tested through Vite+, a unified toolchain that wraps Vite, Vitest, Oxlint, and Oxfmt under a single CLI. The study is guided by three research questions:

RQ1 Which risk scores from the initial planning phase require revision based on empirical evidence from the baseline automation runs?

RQ2 What previously undocumented system behaviours are exposed by extending the test suite with edge-case and failure-scenario tests?

RQ3 How does the observed automation coverage and execution profile compare with the targets set in the planning phase?

The paper contributes: (1) a revised risk matrix with evidence-backed score changes, (2) twelve new test cases across all mandatory categories, (3) documentation of three unexpected system behaviours, and (4) a comparative analysis of planned versus actual QA outcomes.

II. Related Work

Test automation maturity. Wang et al. [1] study test automation maturity in open-source projects using continuous integration. Their empirical findings establish that teams operating at maturity level 3 or above — integrated pipelines with coverage gates — detect defects significantly earlier than teams at lower levels. The Circles pipeline operates at level 3: every commit triggers automated tests, quality gates block failed merges, and coverage is recorded per module.

CI quality gates. Yu et al. [2] examine non-functional requirement testing in continuous integration environments. Their multi-case study shows that quality gates combining pass rate, coverage, and static analysis are more predictive of post-release defect density than any single metric. The Circles pipeline enforces five gates: coverage $\geq$ 80 %, zero critical defects, execution time $\leq$ 5 min, 100 % regression success, and zero static analysis violations.

Boundary failures in TypeScript backends. Tang et al. [3] analyse bugs in the TypeScript ecosystem and identify null-field bypass and permissive-parser acceptance as recurring failure patterns in validation and parsing code. Both patterns are confirmed present in the Circles codebase by

the extended tests in this study, providing empirical support for their taxonomy.

TypeScript and defect detection. Bogner and Merkel [4] demonstrate that static typing alone does not eliminate runtime defects in TypeScript applications. Automated test coverage is a complementary signal, particularly for validation logic where type annotations do not constrain runtime values at system boundaries.

Test suite amplification. Brandt and Zaidman [5] study the interplay between automatic test generation and developer-driven exploration. Their empirical work shows that extending an existing suite with targeted amplified cases — focused on boundary conditions and error paths — surfaces defects that the original suite misses. This study follows the same principle: the baseline suite of 21 tests is amplified with 12 cases that probe boundaries identified through code reading and risk re-evaluation.

REST API test generation. Stallenberg et al. [6] show that hierarchical clustering of request patterns significantly improves both coverage and defect detection for REST API endpoints. The finding is directly relevant to the Circles backend: the Hono API controllers currently have zero automated coverage, and generating even a minimal set of typed request tests could expose contract-level regressions before deployment.

Automated test generation landscape. Fontes and Gay [7] conduct a systematic mapping study of 124 publications on machine-learning-driven test generation. Their synthesis shows that input generation and test oracle construction remain the two most active research areas, with unit and API testing as the dominant target levels. The extended suite addresses both by widening input-space coverage for parsing functions and defining explicit expected outputs for each new case.

III. Methodology

A. Risk Re-evaluation

The original risk formula is retained: Risk Score = Probability × Impact, with both axes scored 1–5. Three evidence sources are used to revise scores:

1) **Pipeline runs**: defects found, flaky test count, and per-module coverage from the baseline CI pipeline.
2) **Edge-case findings**: unexpected behaviours exposed by the new test cases.
3) **Coverage gap analysis**: modules at zero coverage increase in score because detectability cannot be assessed without data.

B. Test Case Design

New test cases target previously identified high-risk modules and are assigned to four categories:

TABLE I
Test category mapping

Category	Goal
Failure scenarios	Verify correct error propagation on bad input or exhausted budget
Edge cases	Probe boundary and structural limits of parsing and validation
Concurrency	Confirm isolation of parallel invocations
Invalid input	Reject malformed or semantically incorrect data

All new tests use Vitest via the Vite+ toolchain, consistent with the baseline suite. Asynchronous tests use vi.useFakeTimers() to eliminate timing-dependent non-determinism. The expansion strategy follows the developer-centric amplification model of Brandt and Zaidman [5]: cases are derived from reading the implementation rather than generated automatically, targeting inputs that lie on or just outside the boundaries enforced by each function's guard conditions.

C. Quality Gate Evaluation

Five quality gates defined in the baseline automation study are re-evaluated:

TABLE II
Quality gate status

Gate	Metric	Threshold	Status
QG01	High-risk module coverage	≥ 80 %	Partial
QG02	Critical defects on trunk	0	Pass
QG03	Test suite execution time	≤ 5 min	Pass
QG04	Regression success rate	100 %	Pass
QG05	Static analysis violations	0 major	Pass

QG01 is partially met: the four automated modules each reach 100 %, but three high-risk modules (API controllers, workflow orchestration, auth middleware) remain at 0 %.

IV. Preliminary Results

A. Revised Risk Matrix

TABLE III
Revised risk scores. Score = Probability × Impact (1–5 each).

Module	Baseline	Revised	Δ
Spotify OAuth & session mgmt	20	20	0
Data ingestion pipeline	20	22	+2
Dashboard stats & filtering	15	15	0
API endpoint correctness	12	14	+2
Friend feed & social features	9	9	0
Playlist management	6	6	0
AI discovery tools	6	6	0
Time Machine view	4	4	0
Static UI components	2	2	0

Two scores increased. The **data ingestion pipeline** rises from 20 to 22 because the extended tests confirm two silent data-quality failure modes (see Section IV-C). The **API endpoint correctness** module rises from 12 to 14 because it remains at 0 % coverage and code review confirms that the auth middleware throws HTTP 401 on missing session only, with no payload schema validation at the controller level.

B. Automation Evidence

Test failures. No failures were observed in any CI run. All 33 tests pass on every push to trunk.

Flaky tests. None detected. Fake timers eliminate real-delay dependencies in all asynchronous tests.

Coverage. Four library modules reach 100 % line coverage. Three integration-layer modules remain at 0 %. Overall high-risk coverage is 4/7 = 57 %, below the QG01 threshold of 80 %.

Execution time. The backend unit suite completed in 206 ms after the suite extension, compared with 95 ms in the baseline run. The increase is proportional to the number of new tests and well within the 5 min gate.

C. Unexpected System Behaviours

Three behaviours were not predicted by the initial risk model:

F-01 — Null album-artist bypass. isValidEntry does not check master_metadata_album_artist_name. Records with a null artist field pass validation and are ingested, producing incomplete statistics on the dashboard without any error signal. Test TC-EXPORT-EDGE-01 confirms this.

F-02 — Permissive URI parser. trackIdFromUri extracts the third colon-delimited segment regardless of the resource type prefix. A Spotify episode URI (spotify:episode:xyz) yields an episode ID rather than a track ID. Test TC-EXPORT-INVAL-01 confirms this. Per Tang et al. [3], permissive-parser acceptance is a known production failure pattern in TypeScript validation layers.

F-03 — Two-byte ZIP check. isZip checks only bytes[0] === 0x50 && bytes[1] === 0x4b (the PK signature prefix). The ZIP specification requires a four-byte local file header (PK\x03\x04). Tests TC-ZIP-EDGE-01 through TC-ZIP-EDGE-03 confirm that two bytes are sufficient to pass the current guard.

These findings are not test failures — the code behaves as written. They are design decisions with unintended consequences that the initial risk model underestimated.

D. New Test Cases

TABLE IV
Extended test cases. All 12 pass.

Test ID	Category	Scenario	Pass
TC-EXPORT-FAIL-01	Failure	ms_played = -1 ⟶ false	✓
TC-EXPORT-FAIL-02	Failure	No-colon URI ⟶ undefined	✓
TC-EXPORT-EDGE-01	Edge	Null album_artist_name ⟶ true (F-01)	✓
TC-EXPORT-EDGE-02	Edge	Empty string ⟶ undefined	✓
TC-EXPORT-EDGE-03	Edge	Base-62 ID round-trip	✓
TC-EXPORT-INVAL-01	Invalid input	Episode URI ⟶ permissive extract (F-02)	✓
TC-RETRY-FAIL-01	Failure	retries=1, rate limit ⟶ 1 attempt	✓
TC-RETRY-CONC-01	Concurrency	Two parallel calls resolve independently	✓
TC-RETRY-INVAL-01	Invalid input	Non-Error rejection ⟶ immediate rethrow	✓
TC-ZIP-EDGE-01	Edge	2-byte PK array ⟶ true (F-03)	✓
TC-ZIP-EDGE-02	Edge	1-byte array ⟶ false	✓
TC-ZIP-EDGE-03	Edge	1 024-byte PK-prefixed array ⟶ true	✓

E. Metrics Summary

TABLE V
QA metrics across study phases

Metric	Planning	Baseline	Extended
Total automated tests	9	21	33
Backend suite time	–	95 ms	206 ms
Test pass rate	100 %	100 %	100 %
Flaky test rate	–	0 %	0 %
CI pipeline pass rate	100 %	100 %	100 %
High-risk module coverage	15 %	67 %	57 %
Critical defects found	0	0	0
Unexpected behaviours found	–	0	3

The overall high-risk coverage decreases from 67 % to 57 % between the baseline and the extended study. This is not a regression: the denominator expands because three previously untracked integration-layer modules (API controllers, workflow orchestration, auth middleware) are now included in scope.

V. Comparative Analysis

A. Planned vs Actual

TABLE VI
Planned vs observed QA outcomes

Aspect	Planned	Observed
Unit coverage target	≥ 80 % on src/lib/	100 % on 4 modules; 0 % on 3 others
E2E coverage	Auth, dashboard, API flows	2 tests — home page only
API integration tests	Planned in first iteration	Not implemented after two iterations
CI pass rate	100 % on every PR	100 % – met
Flakiness rate	< 5 % over 30 runs	0 % – exceeded target
Execution time	< 5 min	206 ms – well within target
Risk model accuracy	9 modules scored	2 revised; 3 new behaviours found
Total effort	~40 hours	~55 hours across all phases

B. Incorrect Assumptions

Three planning assumptions from the initial risk study did not hold.

Integration tests were assumed straightforward. Controller-level tests depend on Better Auth session handling, which requires a live database or a carefully constructed mock. This complexity was underestimated, causing both subsequent iterations to defer this work.

The entry validator was assumed complete. isValidEntry was expected to validate all fields necessary for accurate statistics. Tests TC-EXPORT-EDGE-01 and TC-EXPORT-INVAL-01 show that album_artist_name is not checked and non-track URI types are not rejected.

ZIP validation was assumed standard. The implementation was expected to check the full four-byte local file header. It checks only two bytes (F-03).

C. Missing Scenarios and Design Gaps

Two structural weaknesses are identified in the current automation design.

No integration-level tests. The suite covers pure functions effectively but has no tests for HTTP handler behaviour, database interaction, or background workflow execution. A regression introduced at the controller layer would not be caught before deployment. Stallenberg et al. [6] demonstrate that even lightweight clustering of typed HTTP request patterns achieves substantially higher endpoint coverage than ad-hoc manual testing; applying a similar approach to the Circles Hono controllers is a concrete next step.

No expanded E2E coverage. The Playwright configuration and CI workflow exist, but only two home-page tests are implemented. The OAuth callback, dashboard rendering, and file import flows are all untested end-to-end [1].

VI. Discussion

The risk-first automation strategy proved effective for the library layer. Targeting four high-risk pure functions first produced a stable, fast, and deterministic suite in 206 ms. The unified Vite+ toolchain eliminated per-file configuration and ensured identical test behaviour in CI and local environments.

The edge-case expansion uncovered three design-level observations (F-01, F-02, F-03) that the initial risk model did not predict. These observations map directly to the failure patterns documented by Tang et al. [3]: null-field bypass (F-01) and permissive-parser acceptance (F-02). Both findings support raising the data-ingestion risk score from 20 to 22.

The primary outstanding gap is the absence of integration and E2E tests. Both categories were planned in the initial phase and deferred across two subsequent iterations. The recommended next step is to introduce Hono's test client (app.request()) for controller tests, which does not require a running server, and to add at least one E2E test for the OAuth flow.

QG01 (coverage ≥ 80 %) remains unmet at 57 %. The threshold is not too strict: it correctly identifies the gap. The failure is due to insufficient test scope, not poor code quality. QG02 through QG05 all pass, confirming that the automated scope is stable and the pipeline functions correctly.

Specific improvements for the next iteration: restrict trackIdFromUri to spotify:track:* URIs; add album_artist_name validation to isValidEntry; strengthen isZip to verify all four header bytes; consolidate extended spec files with the baseline spec files.

References

[1] Y. Wang, M. V. Mäntylä, Z. Liu, and J. Markkula, "Test Automation Maturity Improves Product Quality: Quantitative Study of Open Source Projects Using Continuous Integration," *Journal of Systems and Software*, vol. 188, p. 111259, 2022, doi: 10.1016/j.jss.2022.111259.

[2] L. Yu, E. Alégroth, P. Chatzipetrou, and T. Gorschek, "Automated NFR Testing in Continuous Integration Environments: A Multi-Case Study of Nordic Companies," *Empirical Software Engineering*, vol. 28, no. 144, 2023, doi: 10.1007/s10664-023-10356-1.

[3] T. Tang, S. Alimadadi, and N. Sumner, "From Logic to Toolchains: An Empirical Study of Bugs in the TypeScript Ecosystem," in *Proceedings of the 23rd International Conference on Mining Software Repositories*, ACM, 2026, pp. 1–11. doi: 10.1145/3793302.3793379.

[4] J. Bogner and M. Merkel, "To Type or Not to Type? A Systematic Comparison of the Software Quality of JavaScript and TypeScript Applications on GitHub," in *Proceedings of the 19th International Conference on Mining Software Repositories*, ACM, 2022, pp. 658–669. doi: 10.1145/3524842.3528454.

[5] C. Brandt and A. Zaidman, "Developer-Centric Test Amplification: The Interplay Between Automatic Generation and Human Exploration," *Empirical Software Engineering*, vol. 27, no. 4, p. 96, 2022, doi: 10.1007/s10664-021-10094-2.

[6] D. Stallenberg, M. Olsthoorn, and A. Panichella, "Improving Test Case Generation for REST APIs Through Hierarchical Clustering," in *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, IEEE, 2021, pp. 117–128. doi: 10.1109/ASE51524.2021.9678586.

[7] A. Fontes and G. Gay, "The Integration of Machine Learning into Automated Test Generation: A Systematic Mapping Study," *Software Testing, Verification and Reliability*, vol. 33, no. 4, p. e1845, 2023, doi: 10.1002/stvr.1845.